Supplementary Material for Excitation-Aware On-Track System Identification at Full Scale: A Simulation Study

This document holds material moved out of the main paper: the residual model's physics-augmented inputs and objective, the temporal residual and the exact reduction of its cost, the per-phase identification timing, the axis-by-axis comparison with the baseline pipeline as published, and the supporting result figures. Nothing in the main paper's argument depends on it. Section, equation and table numbers of the form *Section V-D* refer to the main paper.

I. Residual Model: Physics-Augmented Inputs and Objective

A. Physics-augmented inputs: the slip angles

The baseline residual reads $\mathbf{z} = [v_x, v_y, \omega, \delta]$ and predicts $[e_{v_y}, e_\omega]$. The quantity it corrects, however, reaches the plant only through the slip angles of (3) of the main paper: the tire law (4) there is a function of (α, F_z) and of nothing else in $\mathbf{z}$, so the target e is, to the extent the single-track structure holds at all, a function of $(\alpha_f, \alpha_r, F_{z,f}, F_{z,r})$ plus the genuinely unmodelled terms. The network is therefore asked to reconstruct α_f, α_r from $\mathbf{z}$ before it can represent anything.

That reconstruction is the worst part of the map to leave to a small network. Equation (3) of the main paper composes a division by v_x with an arctangent; a one-hidden-layer piecewise-linear network approximates a *ratio* only by tiling the (v_y, ω, v_x) domain, and this vehicle's identification data spans 9.2–26.5 m/s, a $2.9\times$ range of the denominator. Eight LeakyReLU units cannot tile that, so the fitted residual represents the same tire nonlinearity independently at each speed it happened to observe, with nothing tying those representations together. The consequence is specific to this pipeline: stage 2 evaluates the network along a synthetic rollout at a single speed and a steering ramp, i.e. exactly in the region where such a residual has no reason to be consistent, and stage 3 then reads the result as tire force.

Appending α_f, α_r, computed from main-paper (3) with the *measured* geometry of Table I of the main paper, hands the network the two arguments of main-paper (4) directly. Three properties motivated the choice. (i) The speed dependence becomes explicit rather than learned: a residual expressed in α transfers between the recorded lap's speed and the rollout speed by construction, which matters because Section V-B of the main paper deliberately drives those two apart. (ii) It costs 16 weights ($58 \to 74$ parameters) and no new sensor, against the several hundred a network would need to tile the division; augmentation is the cheaper way to buy the same function. (iii) It is compatible with the symmetry argument of Section I-B: α_f, α_r are odd in (v_y, ω, δ), so the mirrored batch mirrors the augmented inputs too and the symmetry penalty remains exact.

The raw four inputs are retained rather than replaced, because the effects the residual exists to absorb (load transfer, actuator dynamics, aerodynamic load, combined slip) are not functions of α alone, and v_x in particular carries the load-dependent part. The geometry used for the augmentation is the same l_f, l_r the fit and the force recovery use, which is why $l_{wb} \equiv l_f + l_r$ is enforced (Section V-E of the main paper): an augmentation computed from a different wheelbase than the one (7) assumes would inject a bias exactly where the network is meant to remove one.

This is the physics-informed idea of [1], [2] applied at the input rather than at the loss, and the placement is deliberate: an input augmentation carries no penalty weight to tune, cannot trade against the data term, and costs nothing at inference, whereas a loss term buys the same structure only softly and only where the data supports it. Deep Dynamics [2] makes the complementary choice of constraining coefficients *inside* the network; here the network is left unconstrained and its input space is made physical instead. The augmentation is available on its own (`physics_inputs`) and combined with the temporal residual (**s4** with `use_physics_inputs`), and is rank-agnostic, so the same code path serves a flat batch and a windowed sequence batch.

B. Physics-informed objective

The optional physics-informed objective [1], [2] adds three penalties to the one-step MSE,

$$\mathcal{L} = \mathcal{L}_{\text{data}} + \lambda_s \mathcal{L}_{\text{steady}} + \lambda_m \mathcal{L}_{\text{sym}} + \lambda_g \mathcal{L}_{\text{smooth}}, \quad (1)$$

with $\lambda_s = 0.1$, $\lambda_m = 0.05$, $\lambda_g = 10^{-4}$. Each answers a specific weakness of training a residual on one lap of ordinary driving.

a) Dynamics consistency: $\mathcal{L}_{\text{steady}}$ substitutes the *corrected* next state $\mathbf{x}_{k+1} = \hat{\mathbf{x}}_{k+1} + e_k$ back into (1)–(2) of the main paper, with the axle forces re-evaluated through (4) there at the corrected slip angles, and penalises the squared lateral and yaw balance residuals. Its purpose is to keep the two residual channels consistent with each other: nothing in the data term prevents an e_{v_y} and an e_ω that individually reduce one-step error while jointly implying an axle force pair no tire could produce, and it is that force pair, not the state prediction, that stage 3 subsequently fits. The term is applied only on a quasi-steady mask ($|\dot{v}_y| < 0.8\,\text{m/s}^2$, $|\dot{\omega}| < 6\,\text{rad/s}^2$), because main-paper (1)–(2) neglect load transfer and transient tire relaxation and therefore hold as written only where those

terms are small; penalising the balance during a transient would ask the network to absorb the model's known omissions rather than the plant's unmodelled dynamics.

b) Left/right symmetry: The vehicle is laterally symmetric, so the true residual map is odd: $e(-v_y, -\omega, -\delta) = -e(v_y, \omega, \delta)$. $\mathcal{L}_{\text{sym}}$ penalises $\|f_{NN}(\mathbf{z}) + f_{NN}(-\mathbf{z})\|^2$ over the sign-mirrored batch. A circuit is rarely balanced in its corner directions, so 30 s of lap data covers one sign of δ far better than the other, and the synthetic sweep of stage 2 is run in one direction only. The mirror supplies the missing half of the input domain from geometry rather than from data, at no data cost; the same mirroring is applied to the recorded trace before windowing (Section II), so the two uses are consistent.

c) Input-gradient smoothness: $\mathcal{L}_{\text{smooth}}$ penalises $\|\partial e/\partial \mathbf{z}\|^2$, a soft Lipschitz bound on the residual. Stage 2 queries the network beyond the steering range it was trained on, and Section V-C of the main paper shows what an unconstrained extrapolation there costs. The two mechanisms are complementary and both are kept: the extrapolation bound of Section V-C of the main paper decides *where* the sweep stops asking, and the gradient penalty decides *how fast* the residual is allowed to grow inside that range.

The nominal-model half of (1), the nominal rollout with the mirrored inputs and the constant terms of the symmetry residual, depends only on quantities fixed for the whole training call and is precomputed once per call (Section II-B).

II. Temporal Residual and Its Cost

A. Temporal residual

The `s4` architecture is an S4D diagonal state-space residual [3] over a sliding window of $W = 20$ samples (0.7 s at $T_s = 0.035$ s). The length is a configured value rather than a derived one: it spans roughly 30 times the time constant of the model's fastest lateral mode at the identification speeds, so every window contains the transient the residual has to reproduce. Windows are cut so that none straddles the seam between the recorded trace and its mirrored copy. Each block carries $H = 4$ independent single-input single-output channels of complex state dimension $N = 4$, initialised in the S4D-Lin form ($\mathrm{Re}(A_n) = -\frac{1}{2}$, $\mathrm{Im}(A_n) = \pi n$) that approximates the HiPPO-LegS operator [4] in closed form, with a learnable per-channel discretisation step. As stated in Section II of the main paper, the S4-family residual overlaps with concurrent work [5] and is not claimed as a contribution; Section II-B reports what we did change, which is its cost. Figure 1 shows stage 1 in this configuration.

B. Evaluating the temporal residual at CPU cost

A temporal residual is only usable on-vehicle if it fits inside the re-identification period on the hardware the car carries, which is typically a CPU. Profiled as first written, the S4D path cost 125.7 ms per epoch on a GPU and 898 ms per stage-2 rollout, putting a six-iteration identification at 181.4 s, longer than the 30 s data window it consumes. Three changes remove that cost without altering the arithmetic. None is novel as a technique; what matters here is that the resulting pipeline is real-time-plausible on commodity hardware, and that the equivalence was verified rather than assumed.

a) Closed-form kernel instead of a scan: A diagonal A makes the block's impulse response available in closed form,

$$K_l = \mathrm{Re}\big(C\, e^{A\Delta t\, l}\, \bar{B}\big), \qquad l = 0, \ldots, W-1, \tag{2}$$

so a length-W window is one Vandermonde product followed by a causal Toeplitz matrix product, rather than W sequential steps [3]. At $W = 20$ the dense Toeplitz product is cheaper than the FFT convolution of the original S4 [6] and than a parallel associative scan [7]; both are the right choice at sequence lengths orders of magnitude longer than this one. Because the stack's output layer reads only the window's last timestep, that layer contracts the reversed kernel straight onto the window and skips $W - 1$ of its outputs. Cost: 125.7 ms to 26.7 ms per epoch.

b) Loop-invariant physics-loss terms: Under the physics-informed objective, the nominal Pacejka rollout, the mirrored batch and the constant terms of the symmetry residual depend on the nominal model only, which is fixed for the whole training call, so they are precomputed once per call instead of once per epoch. The three forward passes the objective used to make (data, mirror, smoothness) collapse into one pass over a double-length batch, with the smoothness Jacobian taken off the same graph. Cost: 26.7 ms to 16.6 ms per epoch, with the loss value unchanged bit for bit.

c) The rollout belongs on a CPU: Stage 2 is 500 strictly sequential single-sample steps carrying the S4D hidden state, each with a host synchronisation. That is pure kernel-launch latency on a GPU: 898 ms on CUDA against 204 ms on a CPU copy of the same weights. The rollout is therefore run on a CPU copy whatever device training used, following the recurrent path of Fig. 1(c), and only the full-batch training stays on the GPU, where it is 2.4× faster than a CPU. This is the one place where the two evaluation paths of the same system have to agree exactly, and they are pinned by a regression test to $< 10^{-9}$ in double precision.

End to end this takes a six-iteration identification from 181.4 s to 20.9 s (8.7×) on the same synthetic 30 s lap, with the identified Pacejka coefficients agreeing to four decimal places (Table I). We deliberately did *not* touch the knobs that trade accuracy for speed: epoch count, iteration count, early stopping, warm-starting the network from the previous iteration's weights, or dropping the smoothness term. Those change the identified tire and are a modelling decision, not a free speedup.

III. Identification Cost, per Phase

A. Identification cost

Section II-B's claim is that the temporal residual is affordable on the hardware a vehicle carries; Table I is the measurement, on a synthetic 30 s lap and the same six co-identification iterations throughout. The per-stage rows and the end-to-end totals were measured separately and are not a decomposition of each other: the stage rows reconstruct 156.2 s of the 181.4 s before and 21.1 s of the 20.9 s after, so the residual 25.2 s of fixed cost visible before is not visible after. We report both as measured rather than reconciling them by attribution. Every step was verified against the implementation

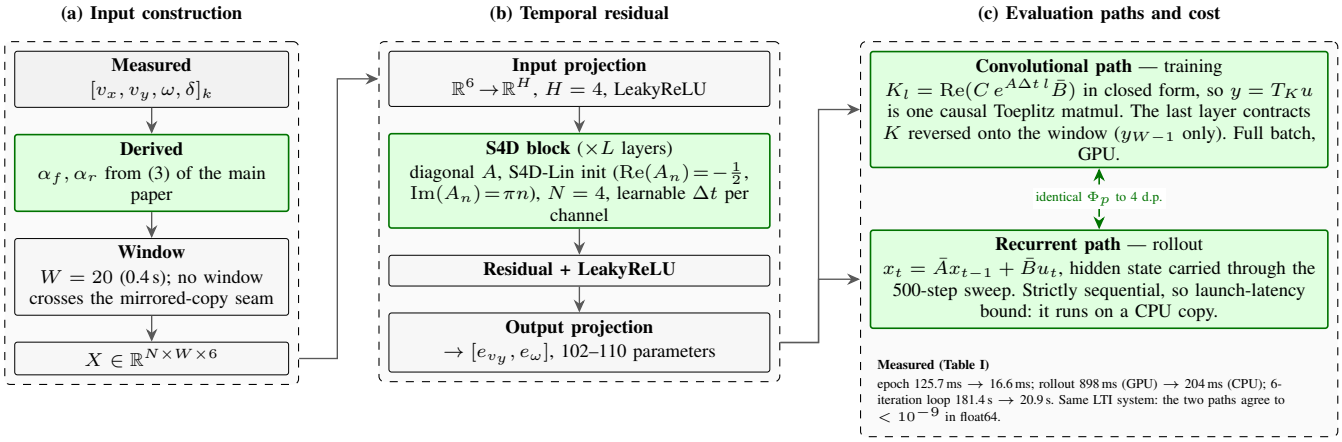

Fig. 1. Stage 1 of Fig. 2 of the main paper with the physics-augmented input vector (Section I-A) and the S4D temporal residual (Section II). Green marks what differs from the baseline MLP residual. (a) The four measured signals are augmented with the two slip angles of main-paper (3) and cut into $W = 20$ sample windows. (b) The residual stack: input projection to H channels, S4D blocks with a diagonal state-transition matrix, and an output projection to $[e_{v_y}, e_\omega]$. (c) The same linear time-invariant system is evaluated two ways, as a causal convolution with the closed-form impulse response for training and as a recurrence for stage 2's stateful rollout, which is what makes the architecture affordable on a CPU without changing a single identified coefficient (Section II-B).

TABLE I
COST OF ONE S4D IDENTIFICATION ON A SYNTHETIC 30 S LAP.

Stage	Device	Before	After
Training epoch: scan → kernel (2)	GPU	125.7 ms	26.7 ms
Training epoch: physics-loss hoist and fusion	GPU	26.7 ms	16.6 ms
Stage-2 rollout, 500 sequential steps	GPU → CPU	898 ms	204 ms
Identification, 6 iterations	mixed	181.4 s	20.9 s

The optimisations are exact: no epoch count, iteration count, early-stopping rule or loss weight was changed, and the identified coefficients agree to four decimal places.

The first three rows are per-stage micro-benchmarks of the named stage; the last row is a separate end-to-end measurement of the whole six-iteration loop and includes everything the micro-benchmarks exclude: data preparation, the per-iteration network re-initialisation and the six Pacejka fits.

The two therefore do not sum: at 1200 epochs and six rollouts the stage rows account for 156.2 s of the 181.4 s before and 21.1 s of the 20.9 s after. The 8.7× is the ratio of the two end-to-end measurements.

it replaced before being kept: the convolutional and recurrent paths of Fig. 1(c) agree to $< 10^{-9}$ in double precision (pinned by a regression test), the rewritten physics-informed loss is bit-identical, and the full six-iteration loop lands on the same Pacejka coefficients to four decimal places, the remaining difference being float32 reordering over 1200 optimiser steps.

Two consequences follow. At 20.9 s the identification fits inside the 30 s re-identification period of Section V-F of the main paper, so the temporal residual is a deployable option rather than an offline one; at 181.4 s it was not. And the device split is not incidental: the stage-2 rollout is strictly sequential and single-sample, so it is 4.4× *faster* on a CPU, while full-batch training over ≈ 3000 windows remains 2.4× faster on the GPU. A pipeline pinned to one device pays one of those two penalties.

IV. THE BASELINE PIPELINE AGAINST THIS WORK

A. Baseline pipeline versus this work

Table II places the pipeline as published [8] beside the pipeline as it stands here, on the axes that changed. The residual model, the data budget, the four-stage loop and the steady-state force recovery are inherited unchanged; what differs is the configuration of the virtual-data step, the regularisation topology, the measurement chain, and the contract at the output. Table III of the main paper's comparison is a subset of these axes: the rollout correction of Section V-B of the main paper is active in both runs of Section VI-C of the main paper, so that section measures the remaining four settings and not the rollout.

V. SUPPORTING RESULT FIGURES

These figures accompany Section VI of the main paper; every metric they show is also in its cross-run comparison table.

REFERENCES

[1] M. Raissi, P. Perdikaris, and G. E. Karniadakis, "Physics-informed neural networks: A deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations," *Journal of Computational Physics*, vol. 378, pp. 686–707, 2019.

[2] J. Chrosniak, J. Ning, and M. Behl, "Deep dynamics: Vehicle dynamics modeling with a physics-constrained neural network for autonomous racing," *IEEE Robotics and Automation Letters*, vol. 9, no. 6, pp. 5292–5297, 2024, preprint: arXiv:2312.04374.

[3] A. Gu, K. Goel, A. Gupta, and C. Ré, "On the parameterization and initialization of diagonal state space models," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2022, preprint: arXiv:2206.11893.

[4] A. Gu, T. Dao, S. Ermon, A. Rudra, and C. Ré, "HiPPO: Recurrent memory with optimal polynomial projections," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2020.

[5] Z. Wu, C. Hu, Y. Wang, L. Xie, and H. Su, "Vision-augmented on-track system identification for autonomous racing via attention-based priors and iterative neural correction," 2026. [Online]. Available: https://arxiv.org/abs/2603.09399

[6] A. Gu, K. Goel, and C. Ré, "Efficiently modeling long sequences with structured state spaces," in *Proc. International Conference on Learning Representations (ICLR)*, 2022, preprint: arXiv:2111.00396.

TABLE II

THE BASELINE PIPELINE [8] AGAINST THIS WORK, ON THE AXES THAT DIFFER.

Axis	Baseline [8]	This work
Platform	1:10, 3.7 kg, $L = 0.32$ m	Full scale, 269.6 kg, $L = 1.53$ m
Rollout speed	$\mathrm{clip}(\overline{v_x}, 2, 4)$ m/s	From the measured lap, floored at 15 m/s
Reachable μ	≈ 2.0 (adequate)	0.43 under the baseline clip; ≥ 1.2 after correction
Sweep amplitude	Fixed 0.4 rad	$1.5 P_{99}(\lvert\delta_{\mathrm{obs}}\rvert)$, clipped to $[\delta_{\min}, \delta_{\max}]$
Regularisation	None; each fit is the next start point	Tikhonov to a *frozen* prior; start point still tracks
Steering signal	Command	Achieved road-wheel angle
Mass, geometry	Configuration file	Measured static wheel loads
Friction prior	—	Per-axle brush fit of (C_α, μ) from IMU and odometry, gated; P_{99} of $\lvert a\rvert/g$ as fallback; always a floor
Solver	Trust-region, single start	Multi-start, plus simplex and global options
Output contract	Per-coefficient box constraints	Gates on derived quantities at the controller interface
Residual objective	One-step MSE	MSE plus quasi-steady balance, left/right symmetry and input-gradient penalties, (1)
Residual inputs	State, command	*inherited* plus axle slip angles (Sec. I-A)
Residual model	MLP, 58 parameters	*inherited* as default; S4D temporal residual shipped
Residual cost	Not reported	S4D identification 181.4 s $\rightarrow$ 20.9 s, exact (Table I)
Data budget	30 s, one controller-free lap	*inherited*
Force recovery	Quasi-steady yaw balance	*inherited*
Ground truth	Steady-state manoeuvre in a large space	Simulator per-wheel force, slip and load

Rows marked *inherited* are unchanged and are listed so that the extent of the modification is unambiguous.

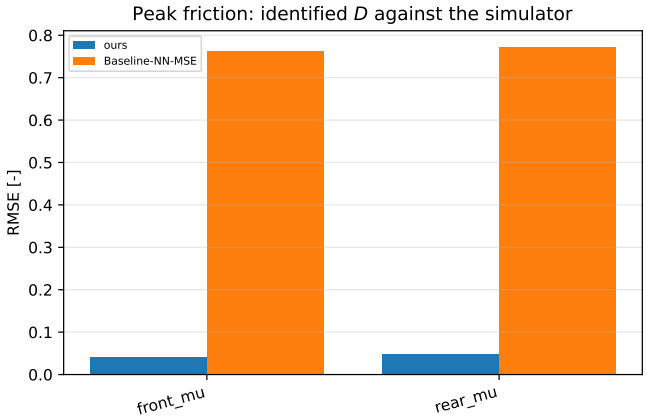

Fig. 2. Peak-friction error per axle against the effective coefficient the simulator reports per wheel ($\mu = 1.05$, static for both runs). The corrected configuration's error is an order of magnitude smaller on both axles (0.041 and 0.048 against 0.762 and 0.772 RMSE); the inherited configuration's error is not a mis-estimate but a coefficient parked on its upper bound.

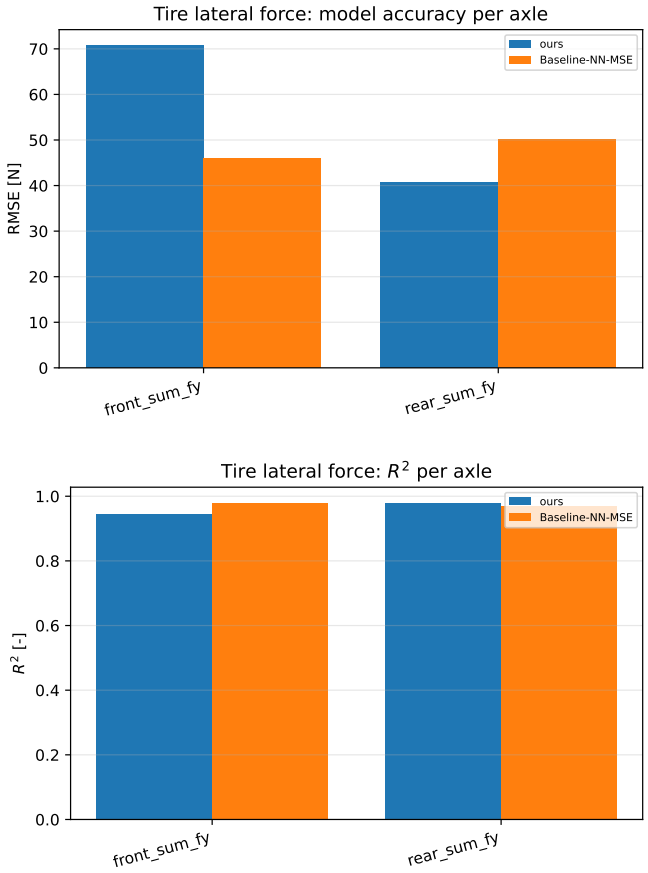

Fig. 3. Axle lateral force against the plant's per-wheel telemetry, per axle: RMSE (top) and R^2 (bottom). The ordering reverses between axles, and this is the one family of metrics on which the inherited configuration is better: at the 2.2 ° of slip the lap reaches, a slope deficit costs force and a wrong peak factor costs nothing.

[7] J. T. H. Smith, A. Warrington, and S. W. Linderman, "Simplified state space layers for sequence modeling," in *International Conference on Learning Representations (ICLR)*, 2023.

[8] O. Dikici, E. Ghignone, C. Hu, N. Baumann, L. Xie, A. Carron, M. Magno, and M. Corno, "Learning-based on-track system identification for scaled autonomous racing in under a minute," *IEEE Robotics and Automation Letters*, vol. 10, no. 2, 2025, preprint: arXiv:2411.17508. Code: https://github.com/ForzaETH/On-Track-SysID.

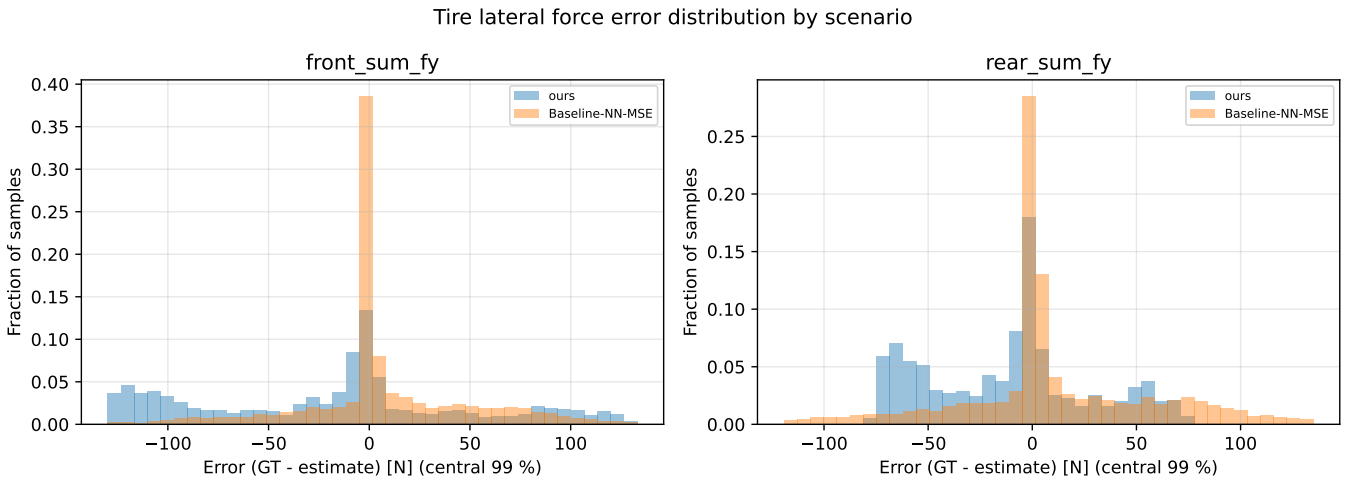

Fig. 4. Distribution of the axle lateral-force error (truth minus estimate), shared bin edges over the pooled central 99 % and normalised to the sample count of each run. Both are centred; the inherited configuration buys its lower front RMSE with the heavier tail (maximum absolute error 215 N against 138 N front, 223 N against 85 N rear).

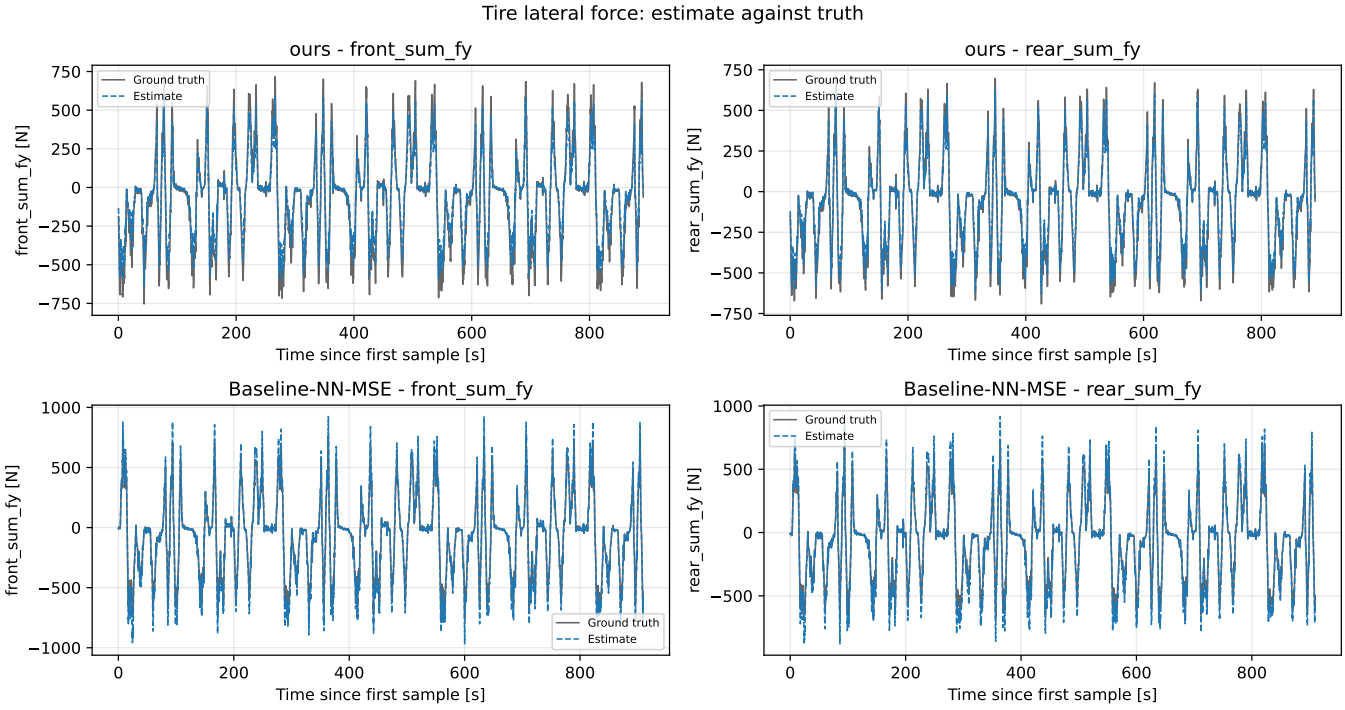

Fig. 5. Axle lateral force over the run, one row per configuration. The two runs are separate laps and share only $t = 0$, so each estimate is drawn against its own ground truth. Where the force is largest each model errs in the direction its initial slope predicts, the corrected configuration short and the inherited one long.

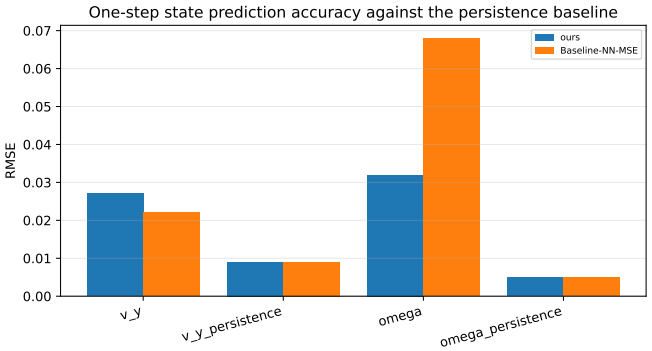

Fig. 6. One-step prediction RMSE for v_y and ω against the persistence baseline of the same run. Neither identified model beats persistence at the 0.035 s model step.

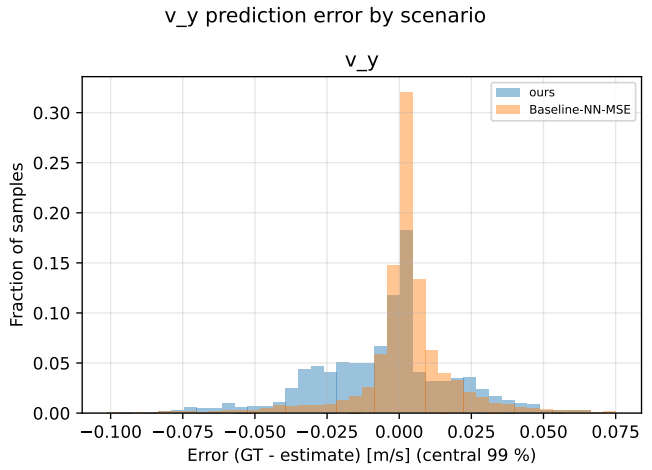

Fig. 7. Lateral-velocity prediction error (truth minus estimate). The two configurations are indistinguishable on this channel (0.027 m/s against 0.022 m/s RMSE); the yaw channel, not this one, is where they differ.

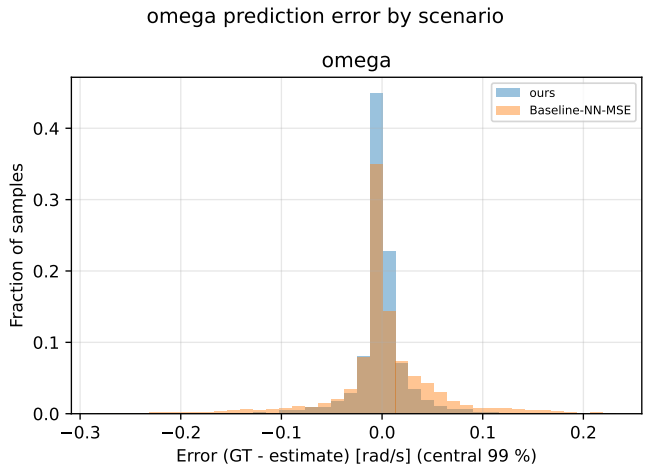

Fig. 8. Yaw-rate prediction error (truth minus estimate). The difference between the configurations is tail mass, not centre.

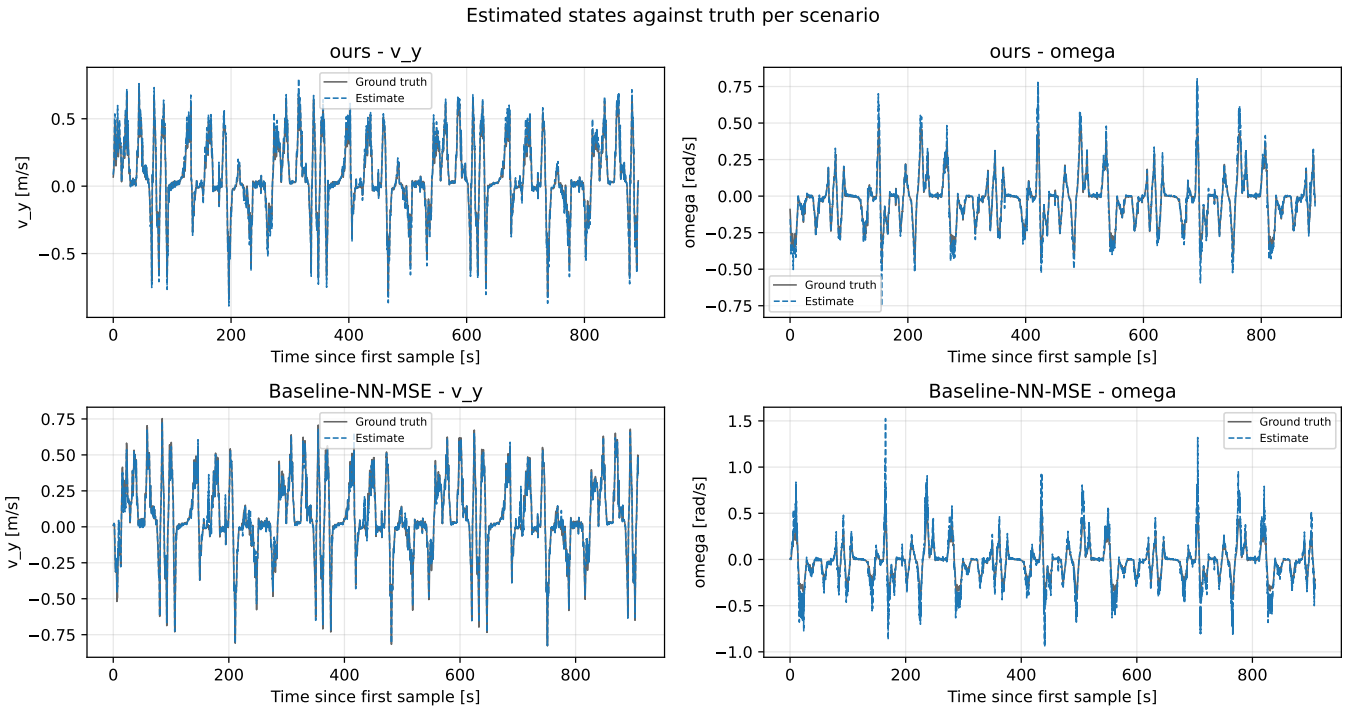

Fig. 9. Estimated states against truth over the run, one row per configuration, 591 s and 564 s of driving. At this compression the traces overlie their ground truth almost everywhere, which is the point: a one-step metric on a smooth racing line is close to uninformative, and the visible departures are the yaw-rate excursions that separate the two configurations in Fig. 8.

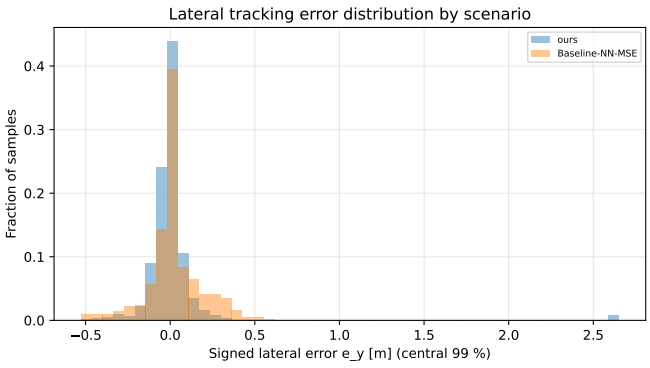

Fig. 10. Signed lateral tracking error over the whole run. The corrected configuration's distribution is narrower about a comparable centre, at sample standard deviation 0.279 m against 0.301 m and mean 0.014 m against 0.039 m, so the improvement is in spread, not in bias.

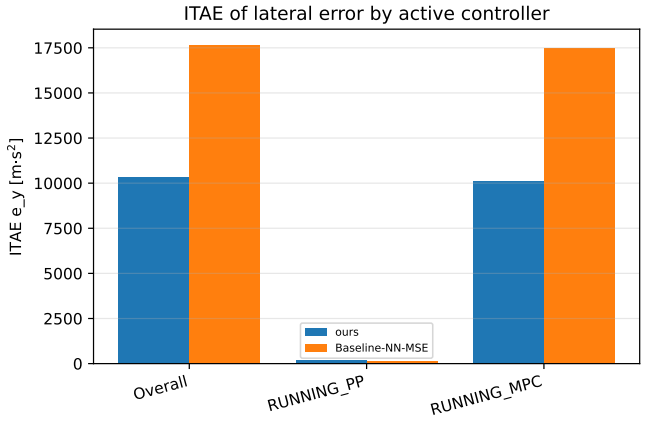

Fig. 11. ITAE of the lateral error, overall and split by the active controller. The pure-pursuit phase is ordered the other way and is two orders of magnitude smaller.